CIS046-3 Software for Enterprise – AY 25/26

Student Name: Kavindu Nethvitha Malshan Piladuwa Loku Bogahawatte

Student Number: 2541047

Student Signature: 

Date: March 2026 (Week 8)

Please answer the questions below, print this form and bring it to the presentation where it will be checked against reality and signed off by the tutor. Once signed off, scan this form and submit on BREO with all other deliverables (video, code). At this stage, Week 8, (at least) a working prototype should be presented plus some ideas on how to continue with the development.

Version Control

How did you develop your code (e.g., what features have ben added after first version)?

The BRAINANA mobile game was developed as a Kotlin-based Android application using Git version control. Key features added include:

- ***Adaptive Theme Engine:*** *Three distinct visual modes (Neural Sync, Super Heroes, Princess Protocol) that dynamically switch puzzle types via different API endpoints*
- ***Leveling & XP System:*** *Progressive ranking system (Initiate → Operative → Architect) with real-time level-up notifications*
- ***Jetpack Compose UI:*** *100% Compose implementation with custom Mesh Gradients for adaptive theming*
- ***Secure Authentication & Cloud Integration:*** *Google Sign In via Firebase Authentication*
- ***Leaderboard System:*** *Cloud Firestore integration for real time global leaderboards and Hall of Fame*
- ***Persona Customization:*** *DiceBear Avatar API integration for dynamic user identity selection*
- ***Gameplay Mechanics:*** *Dynamic difficulty levels (Easy 20s, Medium 12s, Hard 7s) with multiplier-based XP gains*

Event Driven Architectures

What in your code triggers events (e.g., GUI, buttons, timeout …)?

The application implements event-driven architecture through several mechanisms:

1. ***User Input Events:***

- *Button clicks on the "tactical glass keypad" for puzzle solution input*
- *Difficulty selection triggers (Easy/Medium/Hard)*
- *Theme/Neural Vibe toggle buttons trigger dynamic theme changes and API endpoint switches*
- *Navigation events between screens*

2. ***Timer Based Events:***
    - *Countdown timer triggering on each puzzle (20s, 12s, or 7s based on difficulty)*
    - *Timeout events that deduct XP and trigger game state changes*
    - *Level up threshold detection (500 XP intervals) triggering real-time popup notifications*
3. ***Authentication & Network Events:***
    - *Google Sign In completion triggers user profile synchronization*
    - *API response callbacks from Marc Conrad Logic API and DiceBear Avatar API*
    - *Firebase Firestore real time database listeners for leaderboard updates*
4. ***Game State Events:***
    - *Correct answer submission triggers XP increment and progression checks*
    - *Level threshold crossing triggers modal popups*
    - *Theme selection triggers cascading UI and data updates*

All events are managed through MVVM architecture with ViewModel state management and Kotlin Coroutines for asynchronous event handling.

Interoperability

Do you use the API https://marcconrad.com/uob/tomato/api.php?

If no, where in your code do you demonstrate interoperability? What protocols do you use?

Virtual Identity

How did you implement virtual identity in your code? (e.g., Passwords, Cookies, IP Numbers …)

Yes. *The application integrates multiple APIs for interoperability:*

1. ***Marc Conrad Logic API*** *– Primary puzzle generation engine*
    - *Provides mathematical logic puzzles for "Super Heroes (Tomato API)" theme*
    - *Called via OkHttp for high-performance API requests*
    - *Protocols: RESTful HTTP/HTTPS*
2. ***DiceBear Avatar API*** *– User persona customization*
    - *Generates unique avatars for user profile customization*
    - *Integrated for visual identity representation*
    - *Protocol: RESTful HTTP/HTTPS with image caching via Coil*
3. ***Firebase Firestore*** *– Cloud database*
    - *Real-time leaderboard syncing*
    - *User progress and XP persistence across devices*
    - *Protocol: Google Cloud Firestore REST/Websocket*

Technical Implementation:

- *Network requests handled by **OkHttp** for efficient HTTP communication*
- *Image loading managed by **Coil** (Compose Image Loader) for asynchronous rendering*
- *JSON serialization/deserialization for API payloads*
- *Firestore listeners provide real time data updates*

Any other interesting features:

- A dynamic puzzle system where questions are generated through an external API.
- A score system that tracks the player's correct answers.
- A timer feature that makes the game more challenging.
- A simple and interactive mobile interface designed to make gameplay easy and engaging for users.